# How to Evaluate a Model (Part 2)

DSC 340

Week 5, Day 2, March 3

# The Plan

- Discuss a modeling problem
- Review of process (validate/evaluate model)
- Imbalance in datasets/costs
- Confusion matrix
- Precision, recall, and F1 score
- ROC curve and AUC score

# Problem to Model

- CSC 170 students
- Scenario: at midterm predict student “success” at end
- Hypothetical purpose: provide additional support to those “at risk”
- Class labels y: 0 = didn't complete (D/U), 1 = completed (C- or better)
- Suppose data goes back several years for CS1
- Features X: instructor, quiz/lab/test scores, tutoring, prior GPA, etc.

## Discuss

What are some **issues** we may have to resolve?

Suppose we build a model and package says "92% accurate."

**Is that a good result?** Explain.

## Possible Issues

- Changes in the course over time
- New instructors
- Changes in the student body
- How does CSC 160 change things?
- Obtaining all the data is non-trivial
- Dataset imbalanced (more 1 than 0 targets)
- **False positives arguably worse than false negatives**

## Train-test process (one model)

1. Randomly **partition** data into **training** and **test** sets (**train-test split**)
2. **Initialize** classifier
3. Use **training set** to **fit** a model (**learn model parameters**)
4. Use learned model to **predict targets** on **test** set
5. **Evaluate** the model with various **metrics**

TL;DR: We are conducting a scientific experiment.

## Train-test process (multiple models)

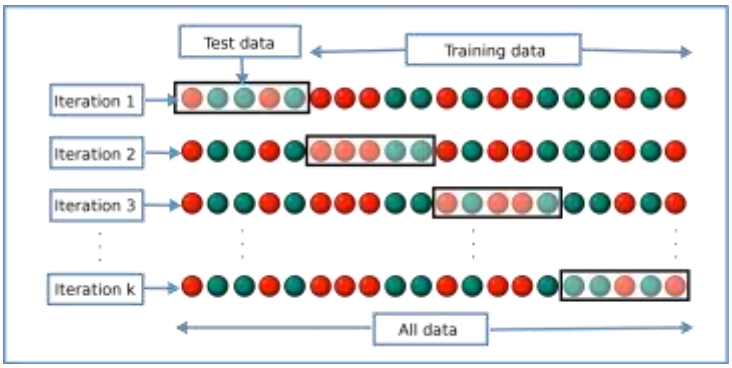
1. Randomly partition data (train-test split)
2. Initialize classifier(s)
3. Use **training set** to **fit models** (learn parameters)
4. Use learned models to **predict targets** on **validation set**
5. Use a **metric** (e.g., F1, AUC) to choose **best model**
6. **Using chosen model**, predict targets on **test (holdout) set**
7. Evaluate chosen model with various metrics

Note: Validation not always static; sometimes rolled up in k-fold CV

# Cross-Validation

Section 5.3

# Cross Validation ($k$-fold C.V.)




Special case: $k = N$

Known as **Leave-One-Out Cross Validation (LOOCV)**

# Cross-Validation

- *k*-folds
  - Single holdout: *k* = 1
- Training/Testing
- Examples:
  - Threefold cross-validation
  - Stratified 10-fold cross-validation
- Still random variation …
- *Repeated* k-fold cross-validation

# Cost of Errors

All errors are equal, but some are more equal than others

## Cost of Errors

- Mastitis detection in dairy cattle
  - Predicts negative: business as usual
  - Predicts positive: remove from production
  - What are the tradeoffs?
- What are the costs of errors?
  - False positive?
  - False negative?
- Google image classification ☹

# Confusion Matrix

| Predicted: Actual: | Cat | Dog |
|---|---|---|
| Cat |   | |
| Dog | |   |

# Confusion Matrix

| Predicted: Actual: | Cat | Dog |
|---|---|---|
| Cat |  CAT | CAT |
| Dog | DOG |  DOG |

# Confusion Matrix

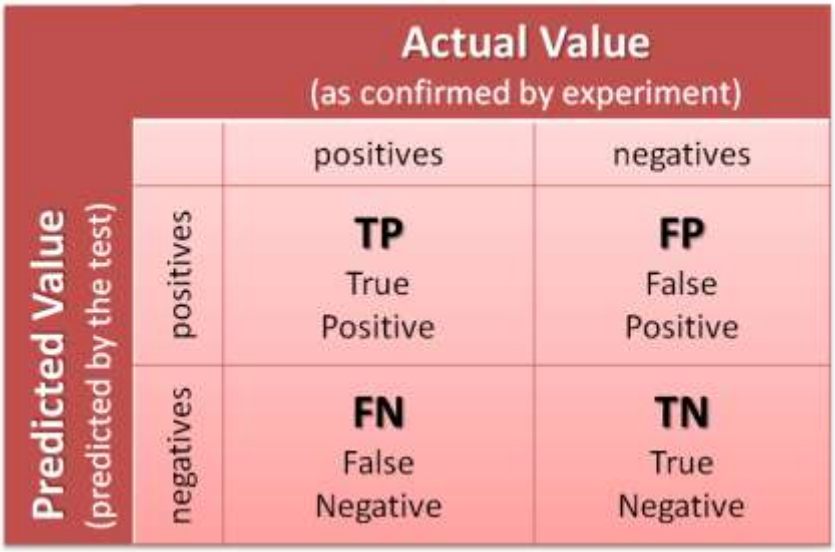
| Predicted: Actual: | Cat | Dog |
|---|---|---|
| Cat |  CAT | CAT |
| Dog |  DOG |  DOG |

# Confusion Matrix

| Predicted: Actual: | Cat | Dog |
|---|---|---|
| Cat |   |   |
| Dog |   |   |

# Confusion Matrix

| Predicted: Actual: | Cat | Dog |
|---|---|---|
| Cat |   |   |
| Dog |   |   |

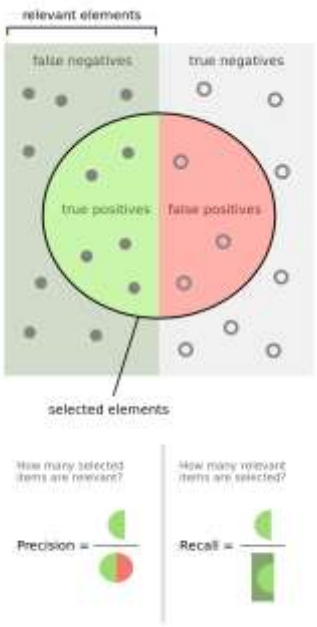



https://www.mathworks.com/matlabcentral/fileexchange/60900-multi-class-confusion-matrix

Benchmark Results - Confusion matrix (Split 90:10)

| | <unclass | Finance | Food | Indices | Metals | OilGas | RawMat | ship | Trading |
|---|---|---|---|---|---|---|---|---|---|
| <unknown> | | | | | | | | | |
| Finance | | 4320 / 175 | 2 | 3 | 1 | 9 | | 1 | 23 |
| Food | | 14 | 271 / 17 | | 1 | 2 | | 4 | 9 |
| Indices | | 10 | 2 | 139 / 5 | | 2 | | | 15 |
| Metals | | 27 | 1 | | 141 / 10 | 6 | 2 | | 10 |
| OilGas | | 38 | | | 1 | 225 / 13 | | | 5 |
| RawMaterials | | 7 | 1 | | | 2 | 39 / 0 | 1 | 1 |
| ship | | 14 | 3 | | 1 | 3 | | 86 | |
| Trading | | 17 | | 6 | 1 | 4 | | 1 | 223 / 1 |

http://www.skilja.de/wp-content/uploads/2015/08/Confusion-Matrix-hierarchic1.png




https://en.wikipedia.org/wiki/Precision_and_recall

# Precision and recall

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- **Did we find all relevant data?**
- *Did we find most of the cat pics?*
- **Emphasizes cost of FP**
- Score between 0 and 1
- Relative to class (can average)

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- **Did we avoid irrelevant data?**
- *Did we avoid lots of non-cats?*
- **Emphasizes cost of FN**
- Score between 0 and 1
- Relative to class (can average)

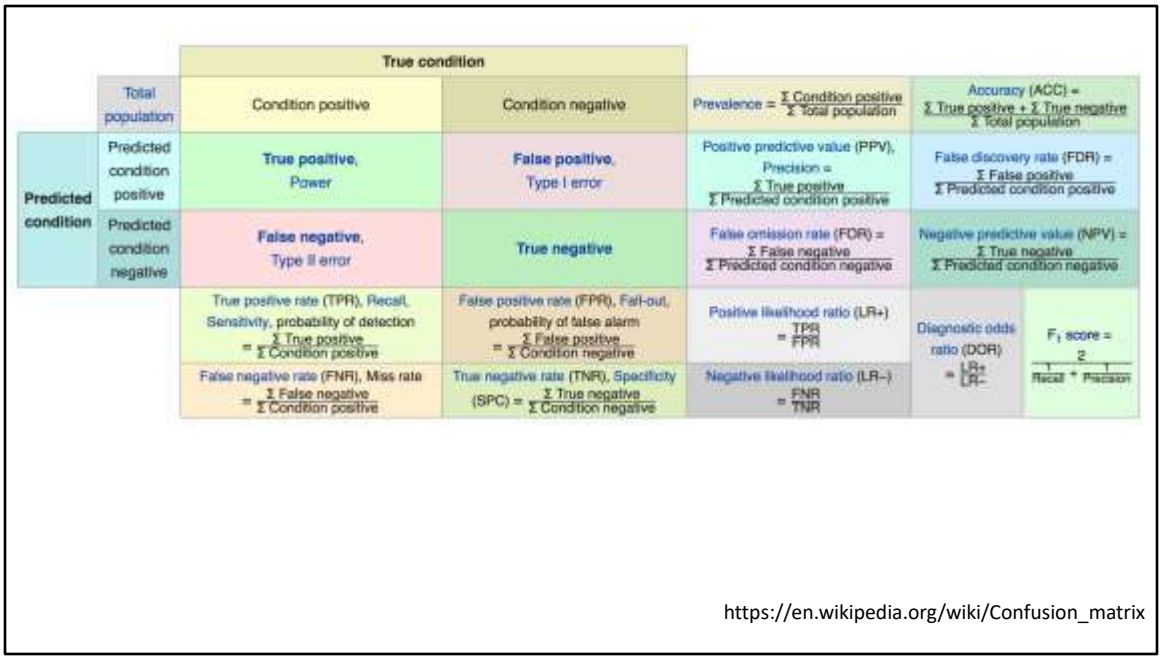
## $F_1$ Score

$$\mathrm{F}_1 \text{ Score} = \frac{2}{\frac{1}{\text{Recall}} + \frac{1}{\text{Precision}}}$$

- Attempt to harmonize (harmonic mean)
- Also relative to class but same number if binary
- Often used as to choose best model if automated
- Still in range [0,1] but not a percent

## Support

- Number of instances actually in each class
- Just a count of true labels, not a ratio
- **Imbalanced data set:** a lot more of one class than other
- If imbalanced, accuracy score can be high, but precision or recall low
- For example, may just predict all negative!




https://en.wikipedia.org/wiki/Confusion_matrix

# Confusion Matrix

| Predicted: / Actual: | Cat | Non-Cat |
|---|---|---|
| Cat | 5   | 2   |
| Non-Cat | 3   | 17   |

# Confusion Matrix

**Recall = 5/(5+3) = 5/8**

| Predicted: / Actual: | Cat | Non-Cat |
|---|---|---|
| **Cat** | 5   | 2   |
| **Non-Cat** | 3   | 17   |

# Confusion Matrix

**Precision = 5/(5+2) = 5/7**

| Predicted: / Actual: | Cat | Non-Cat |
|---|---|---|
| **Cat** | **5**   | **2**   |
| **Non-Cat** | **3**   | **17**   |

# Confusion Matrix

| Predicted: Actual: | Cat | Non-Cat |
|---|---|---|
| Cat | 5 | 2 |
| Non-Cat | 3 | 17 |

$$F_1 = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}} = \frac{2}{\frac{8}{5} + \frac{7}{5}} = \frac{2}{3}$$

## Measures (so far!)

- Error rate
- Precision
- Recall
- $F_1$ score
- Confusion matrix
- Support
  - Number of samples of the true response that lie in that class

# ROC curves

EMC 7.3b

# ROC curve

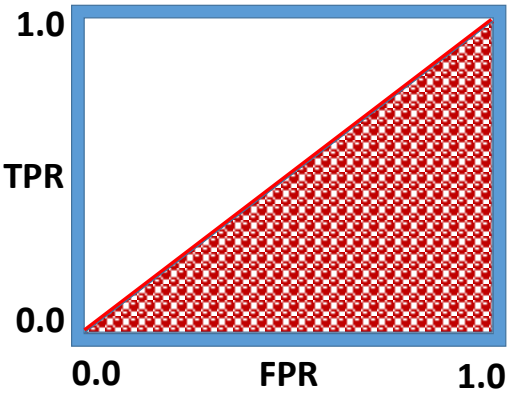
- ROC = "Receiver Operating Characteristic"
- Term from Signal Processing
  - Assumes noisy channel
  - Trade-off between hit rate and false-alarm
- Evaluates performance of classifier based on TP vs FP
  - Ignores class distribution and error costs

# ROC curve

- Vertical axis: "Recall"
  - a.k.a. **True Positive Rate [TPR]**
  - $Recall = TPR = \frac{TP}{TP+FN}$
  - "sensitivity"
- Horizontal axis: **False Positive Rate [FPR]**
  - $FPR = \frac{FP}{FP+TN}$
  - "fall-out" or 1 – "specificity"

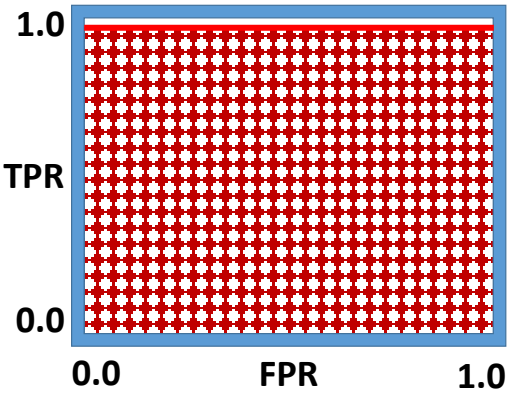
# Worst cases

- Imagine a lazy classifier.
- Classifies everything negative.
  - Then TPR = FPR = 0.
- Classifies everything positive.
  - Then TPR = FPR = 1.
- Just guess (coin flip).
  - For suitable threshold $p$, could still have TRP = FPR.



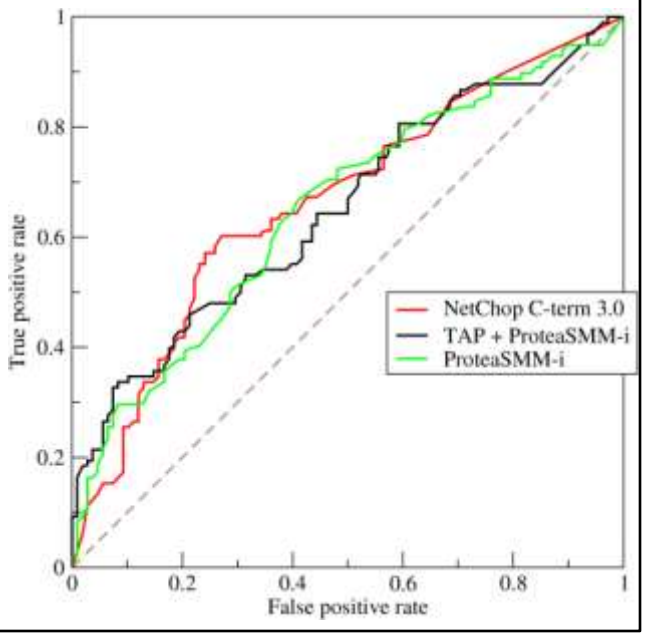
# Best case

- Perfect classifier.
- TPR = 1
- FPR = 0.
- But also:
  - Accuracy = 1
  - P = 1, R = 1, F1 = 1.
- **In reality, it can be difficult to achieve upper-left corner!**



# More realistic

- Example:
  - 3 predictors peptide cleaving in proteasome
  - Source: Wikipedia
- Imagine we had a dial
  - Right = TP goes to 1
  - Left = FP goes to 0
- Lets us control trade-off
  - Sensitivity vs specificity
  - But there is still a trade-off!



# Cylon (or terrorist) detector

- Lots of humanoids in the fleet
- Most are humans, small number are Cylon infiltrators
- Cylon detector (claim: 90% accurate)
  - Will be many false positives
- By setting threshold, can dial it up so it detects all the Cylons
  - Tradeoff: But there are a huge number of false positives
- Or dial it down so we have fewer false positives
  - Tradeoff: Less likely to identify who is a Cylon
- ROC curve lets us visualize this kind of tradeoff

# AUC

- **Area Under the (ROC) Curve**
- Just the area under the **ROC** curve
- Between 0 and 1 (in practice, 0.5 and 1)
- Like F1 score, gives us a number to use in comparing models
- Higher is better

## Jupyter

```
y_pred = model.predict(X_test)
accuracy = 100 * model.score(X_test, y_test))
cm = metrics.confusion_matrix(y_test, y_pred)
y_scores = model.predict_proba(X_test)[:,1] # true pos
auc = metrics.roc_auc_score(y_test, y_scores)
fpr, tpr, threshold = metrics.roc_curve(y_test, preds)
roc_auc = metrics.auc(fpr, tpr)
# and then plot in MPL from data computed above
```

# Extended Example

See Jupyter Notebook how_to_evaluate.ipynb